

Architettura degli Elaboratori

L'architettura della CPU – Gestione dei data hazard



SAPIENZA
UNIVERSITÀ DI ROMA

Davide Polese

davide.polese@uniroma1.it

Special thanks and credits: Alessandro
Checco, Andrea Sterbini, Iacopo Masi,
Claudio di Ciccio

[S&PdC]

4.4 - 4.7



Argomenti

Per la pipeline, lo schema adottato è quello di una **catena di montaggio**

- **Obiettivo: fasi veloci** (periodo di clock = durata della fase più lenta)
- **Ciascuna fase realizza** (localmente a un'istruzione) **un solo task**. Più task possono essere eseguiti in parallelo
- Ciascuna fase **riceve informazioni e segnali di controllo**
- Ciascuna fase **passa alla successiva le informazioni e segnali di controllo**
- I segnali necessari **devono restare stabili** durante tutta la fase

Si devono usare **elementi di stato** (es., flip-flop) che cambiano solo al fronte di salita del clock

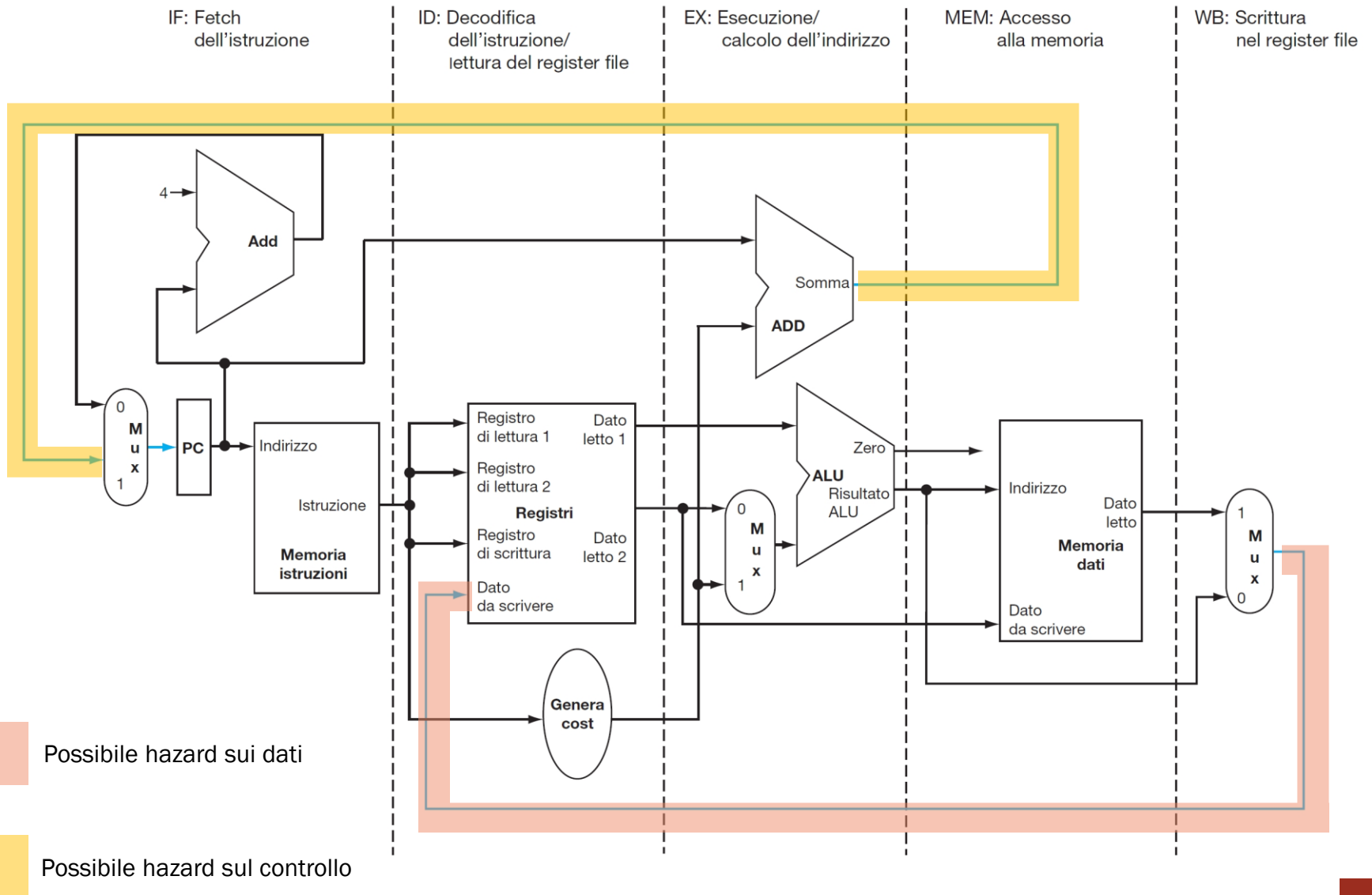
Soluzione: separare ciascuna fase dalla successiva con un **registro**

Architettura della CPU RISC-V con pipeline

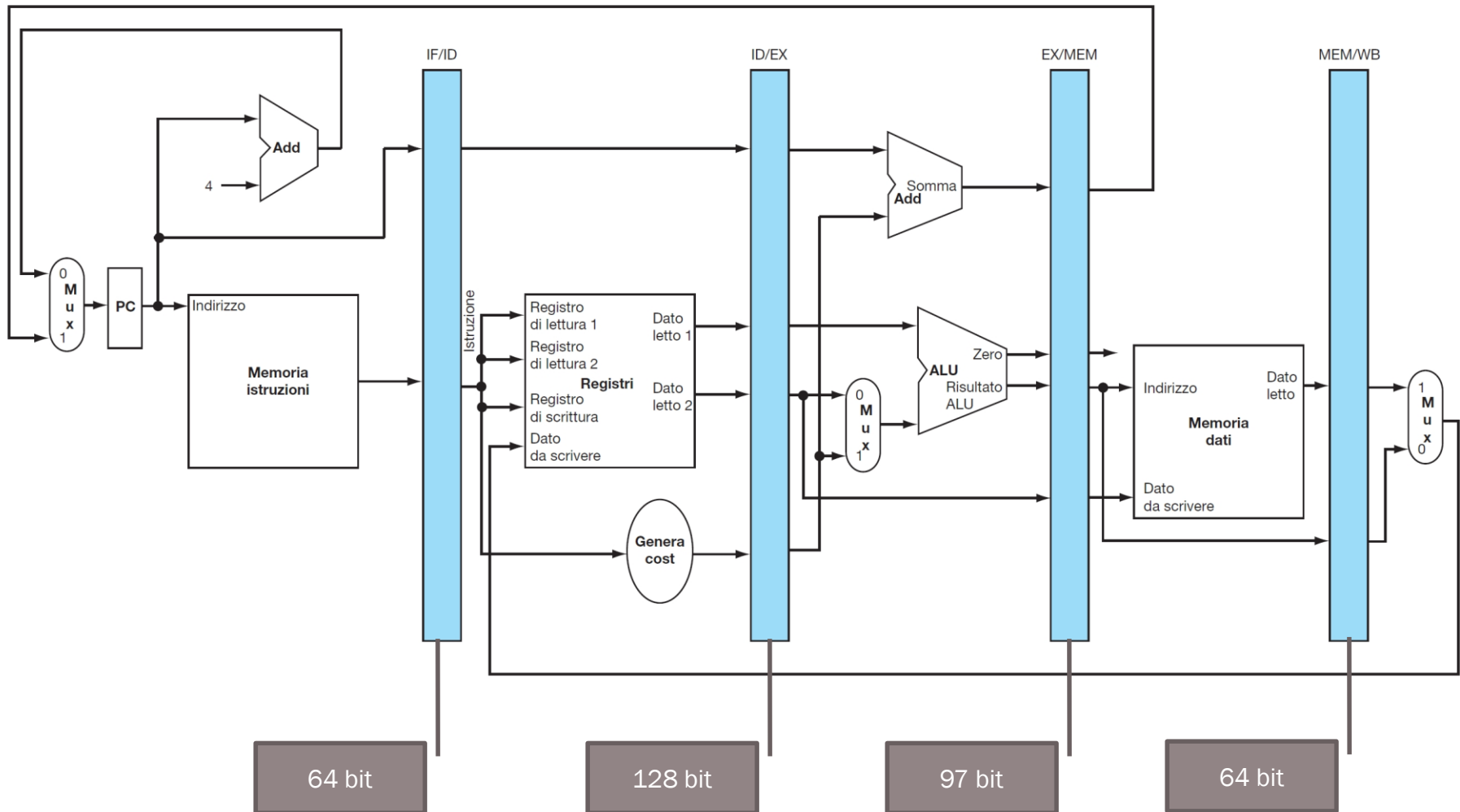


SAPIENZA
UNIVERSITÀ DI ROMA

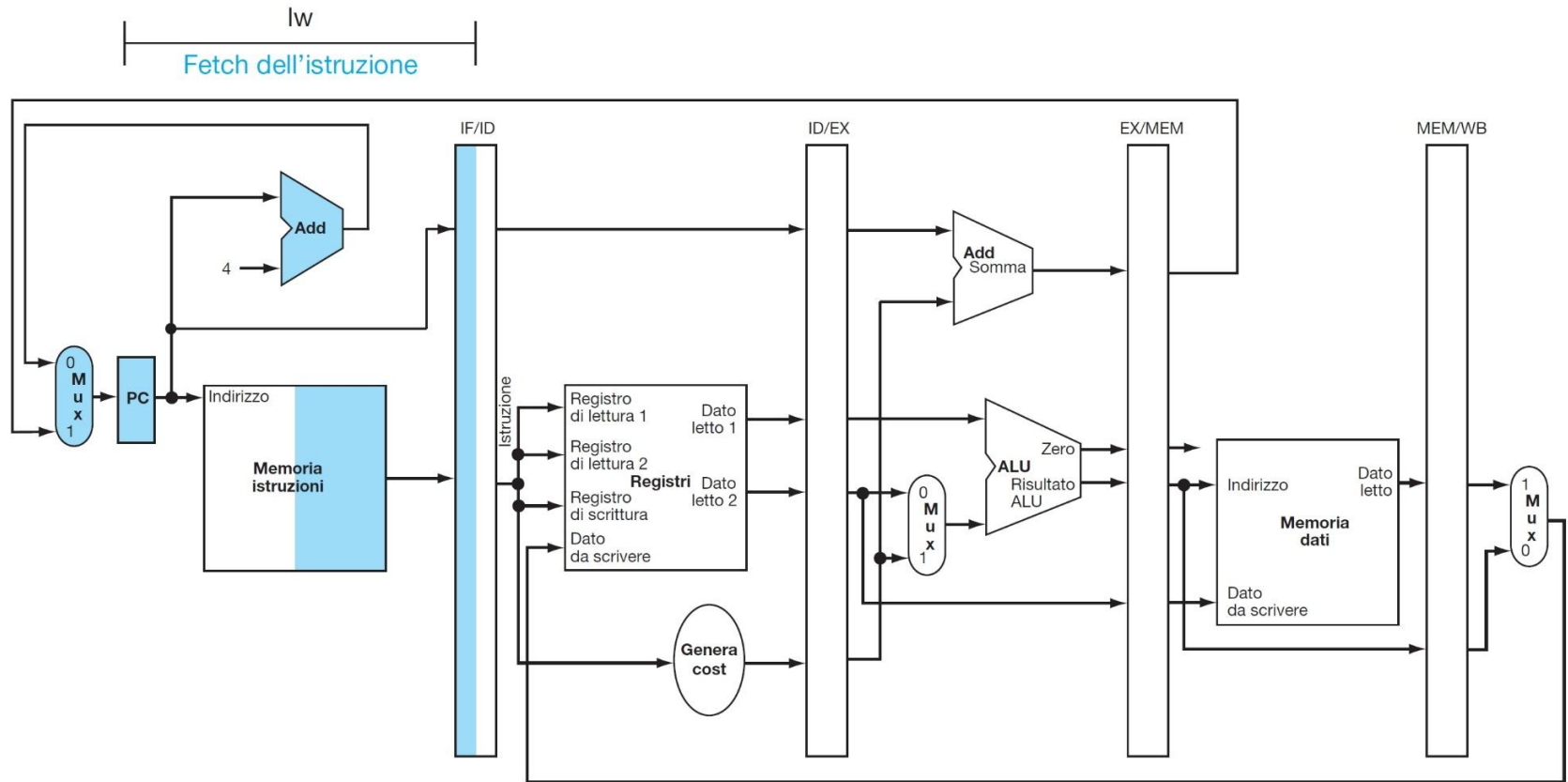
Unità funzionali delle 5 fasi



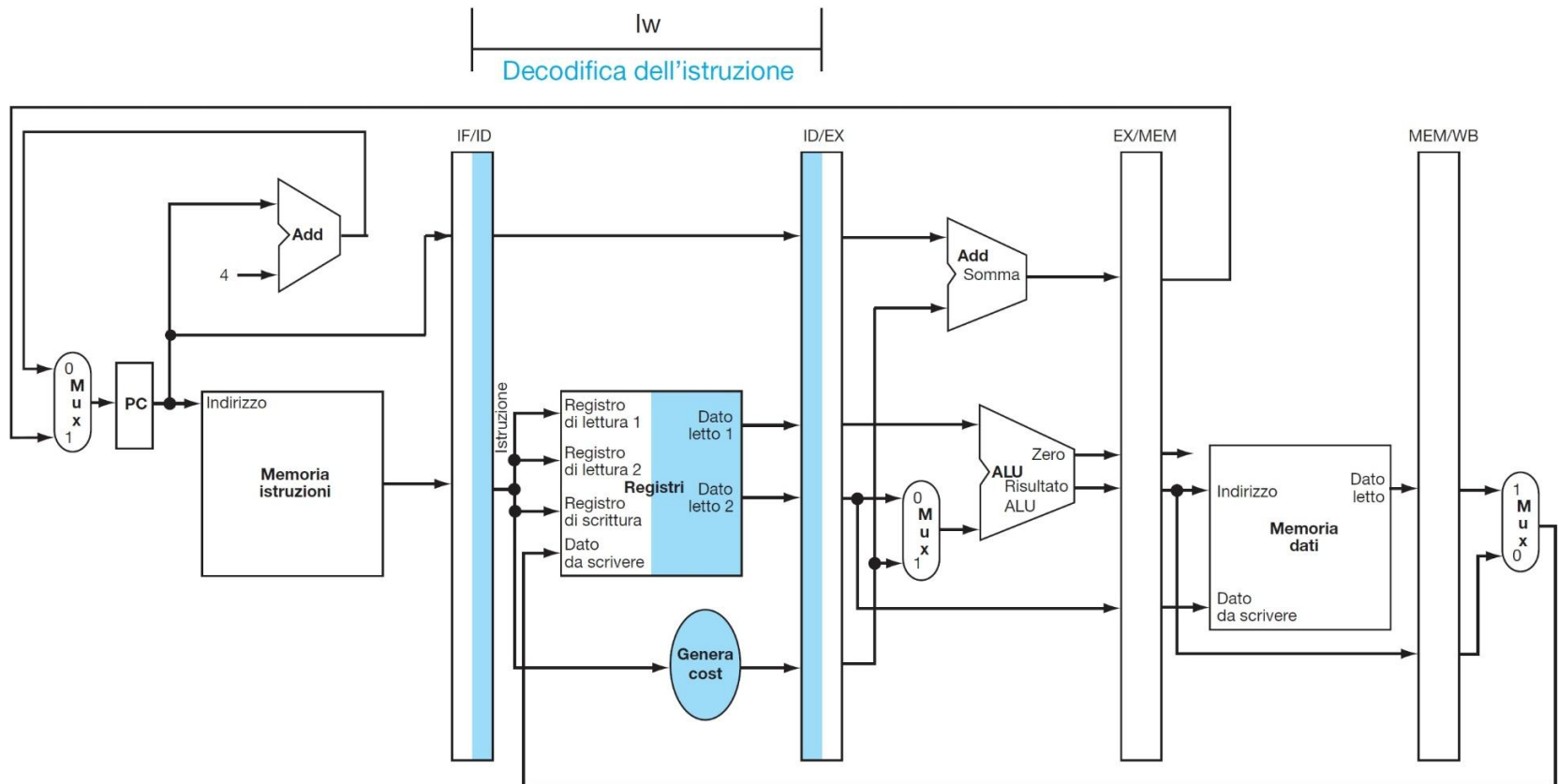
Inseriamo i registri tra le fasi



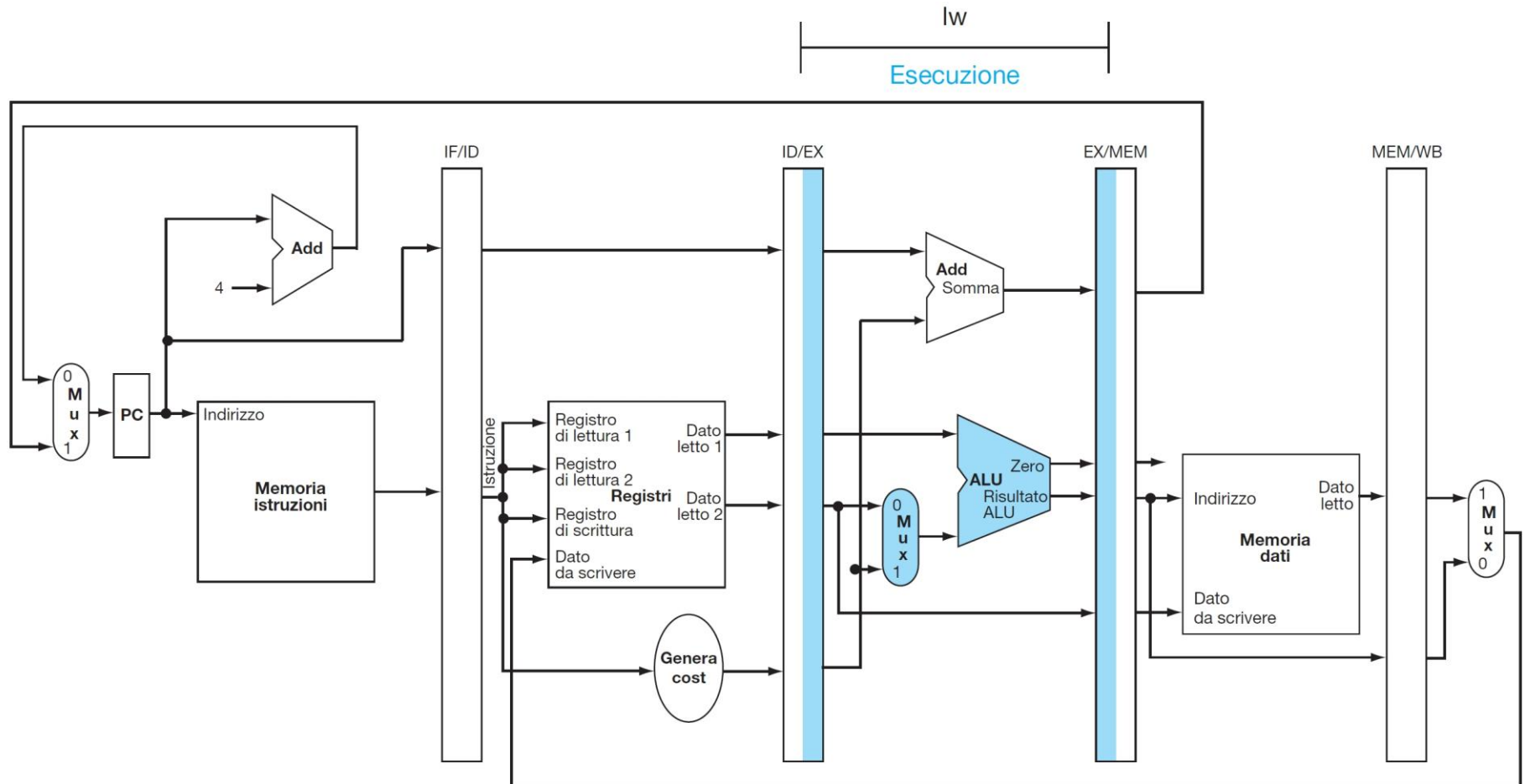
Instruction Fetch



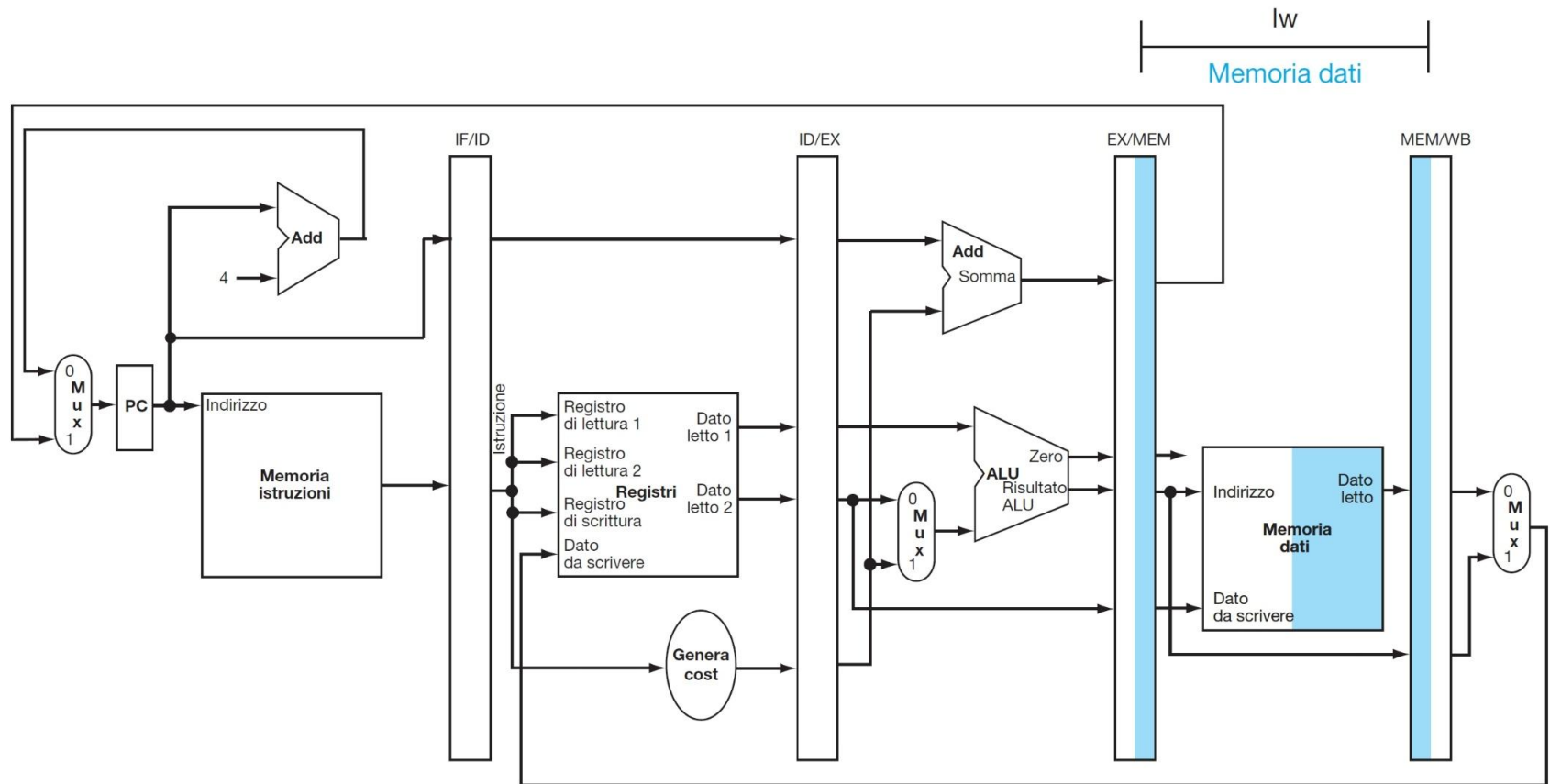
Instruction Decode



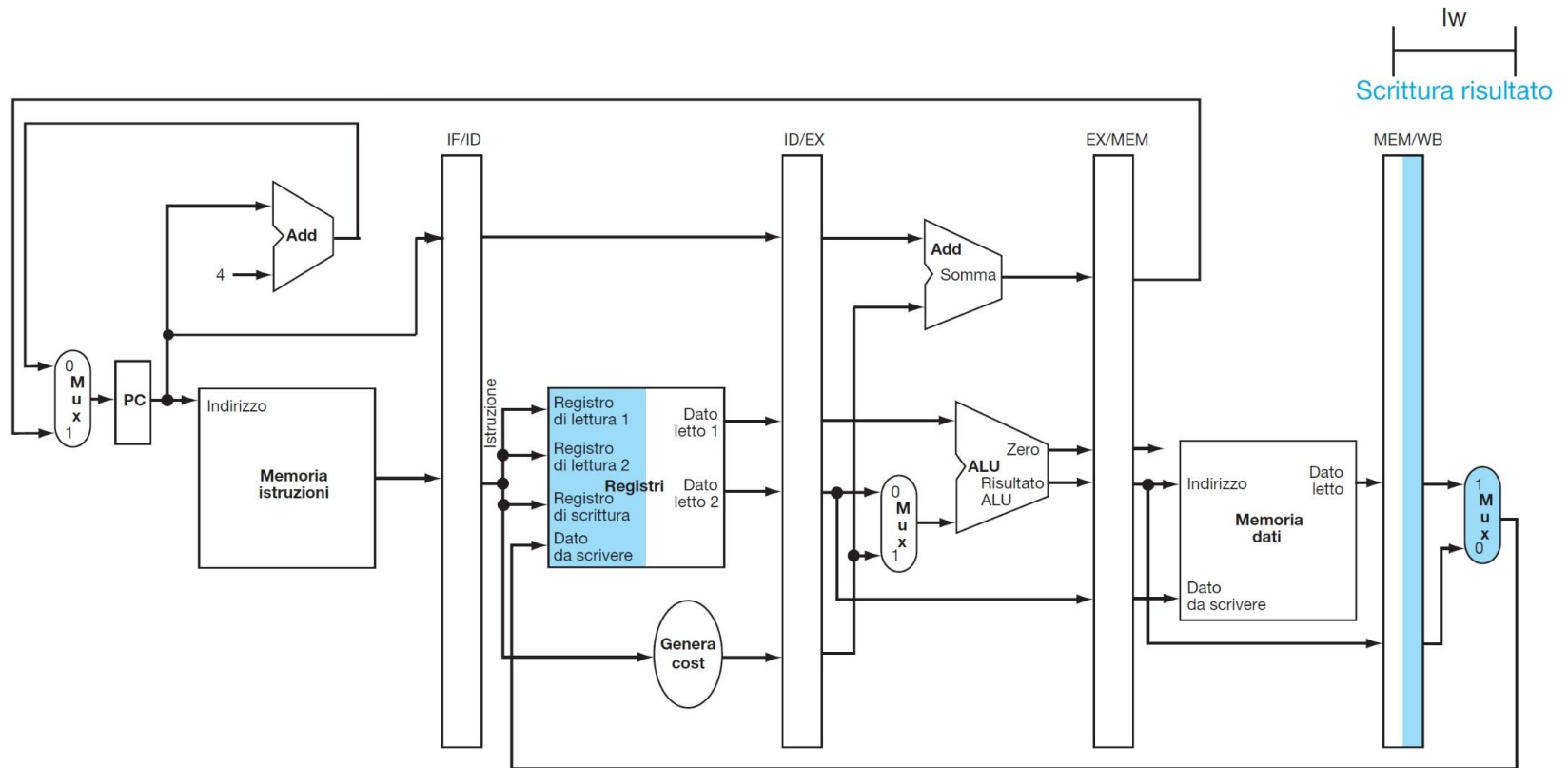
EXecution



MEMory access

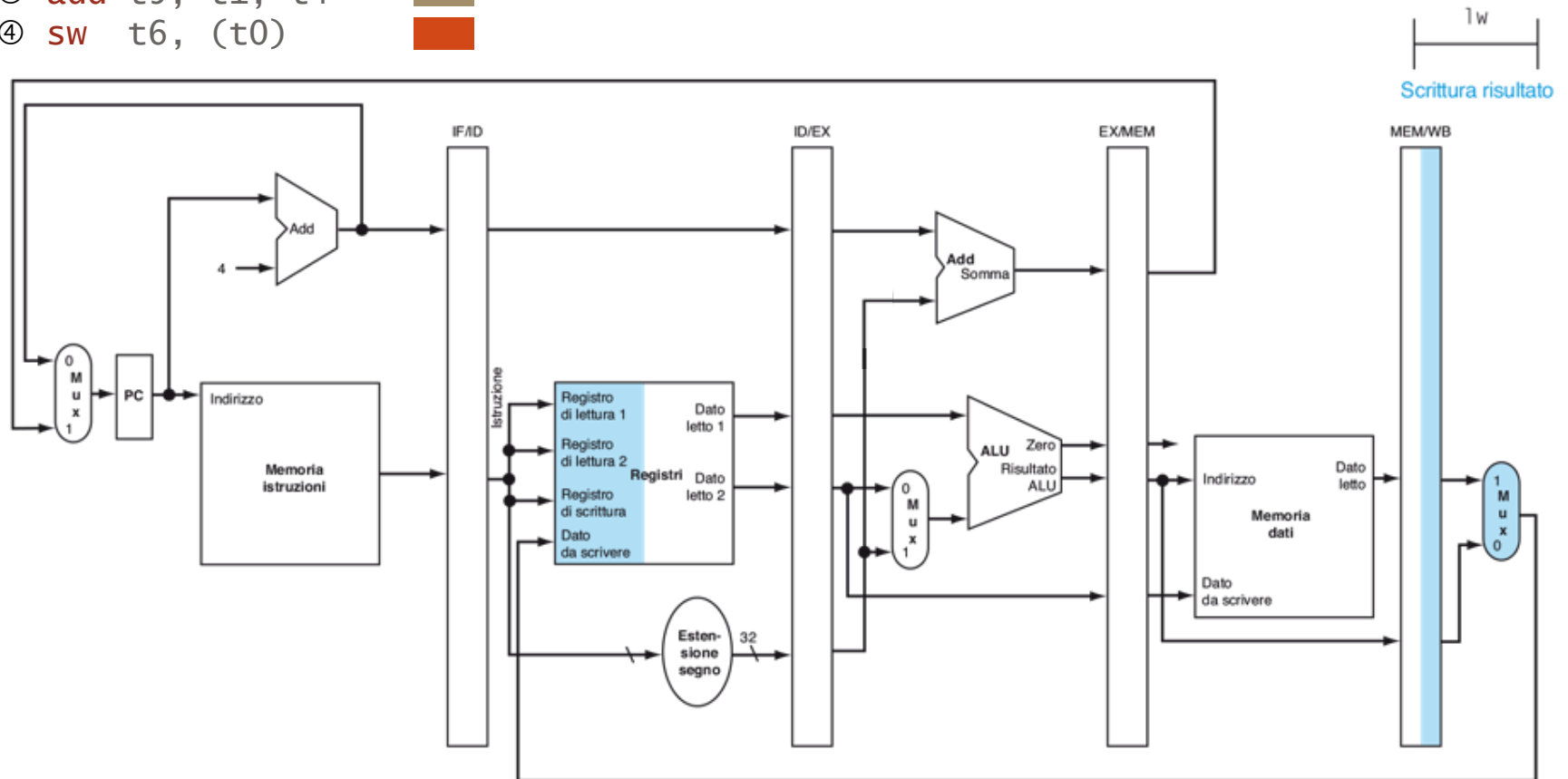


Write Back



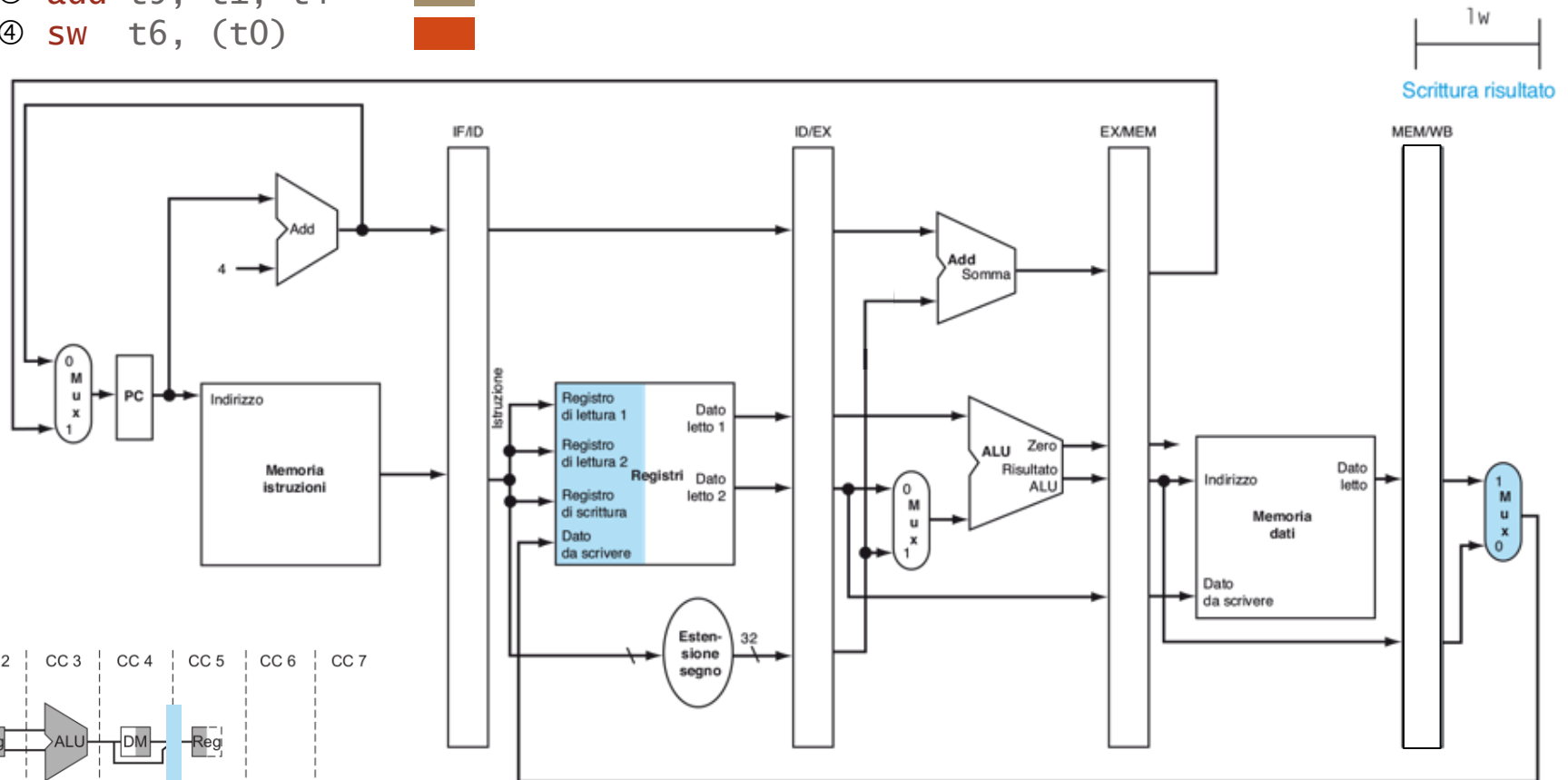
Write Back e altre istruzioni

- ① lw t4, (t0)
- ② add t3, t1, t2
- ③ add t5, t1, t4
- ④ sw t6, (t0)

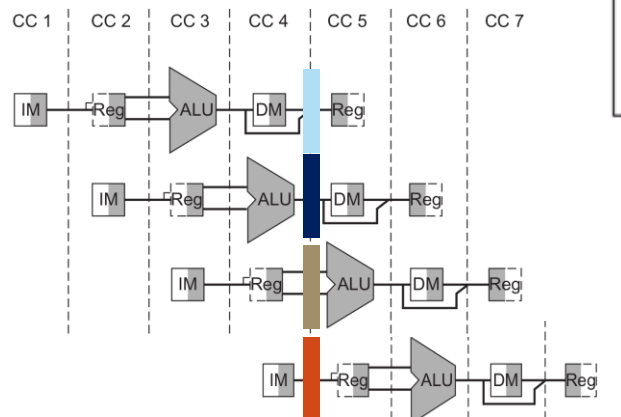


Write Back e altre istruzioni

- ① **lw** t4, (t0)
- ② **add** t3, t1, t2
- ③ **add** t5, t1, t4
- ④ **sw** t6, (t0)



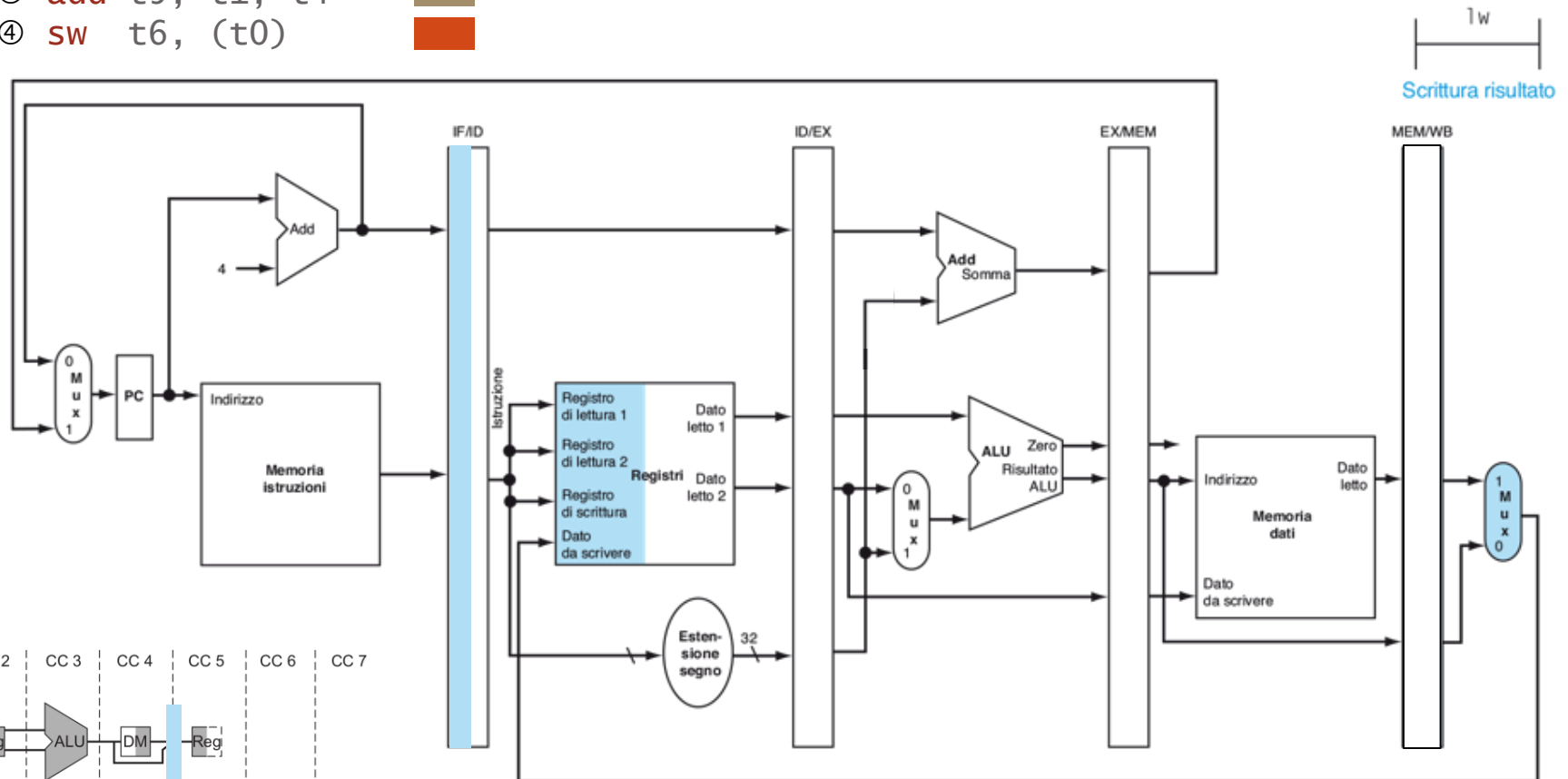
1w
Scrittura risultato



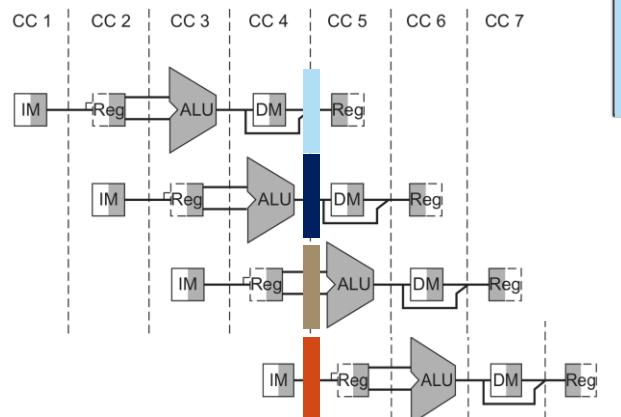
In questo modo, il write-back avviene sul registro sbagliato.
(l'indice del registro di destinazione proviene dalla
istruzione 3 passi dopo)

Write Back e altre istruzioni

- ① **lw** t4, (t0)
- ② **add** t3, t1, t2
- ③ **add** t5, t1, t4
- ④ **sw** t6, (t0)



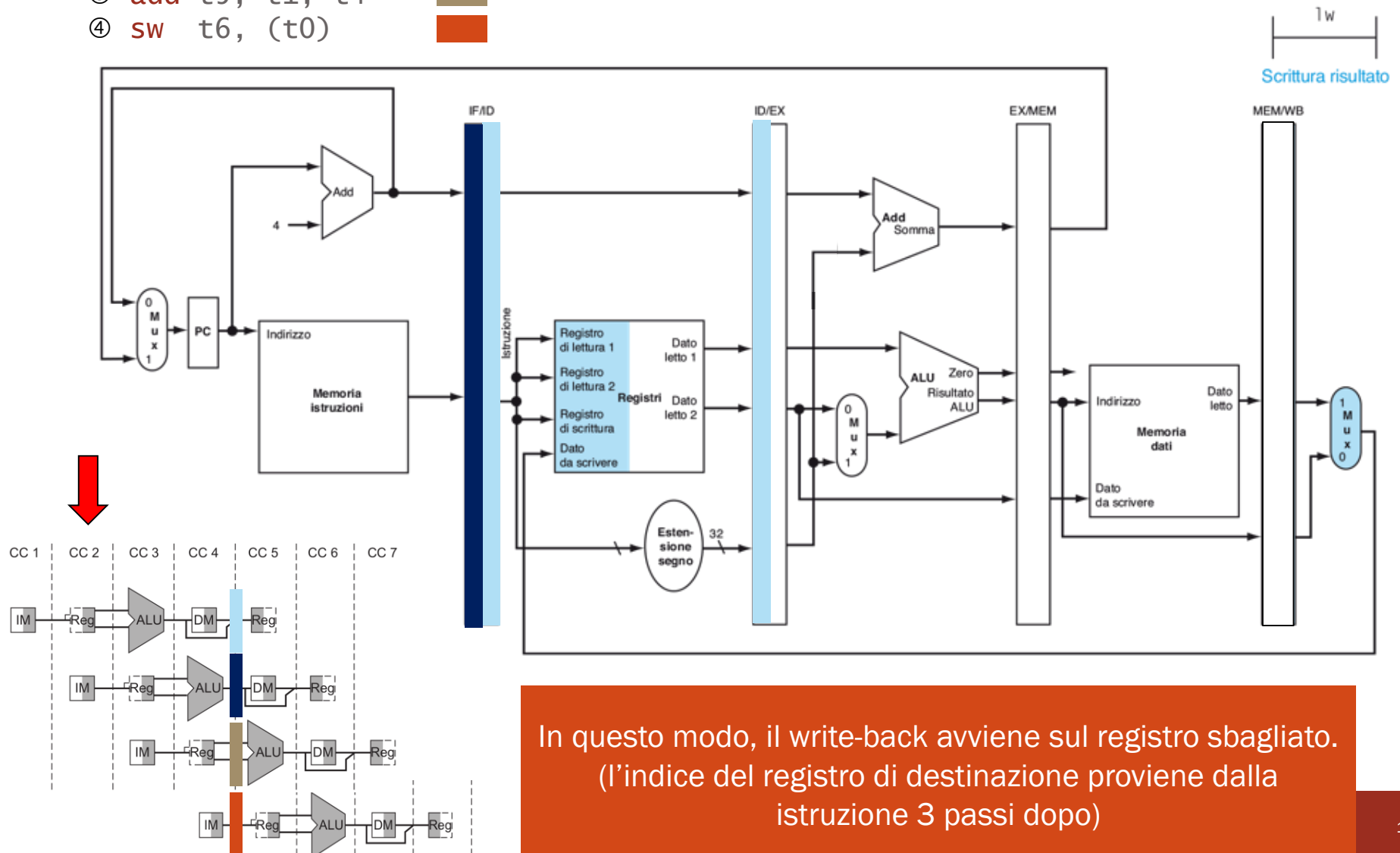
1w
Scrittura risultato



In questo modo, il write-back avviene sul registro sbagliato.
(l'indice del registro di destinazione proviene dalla
istruzione 3 passi dopo)

Write Back e altre istruzioni

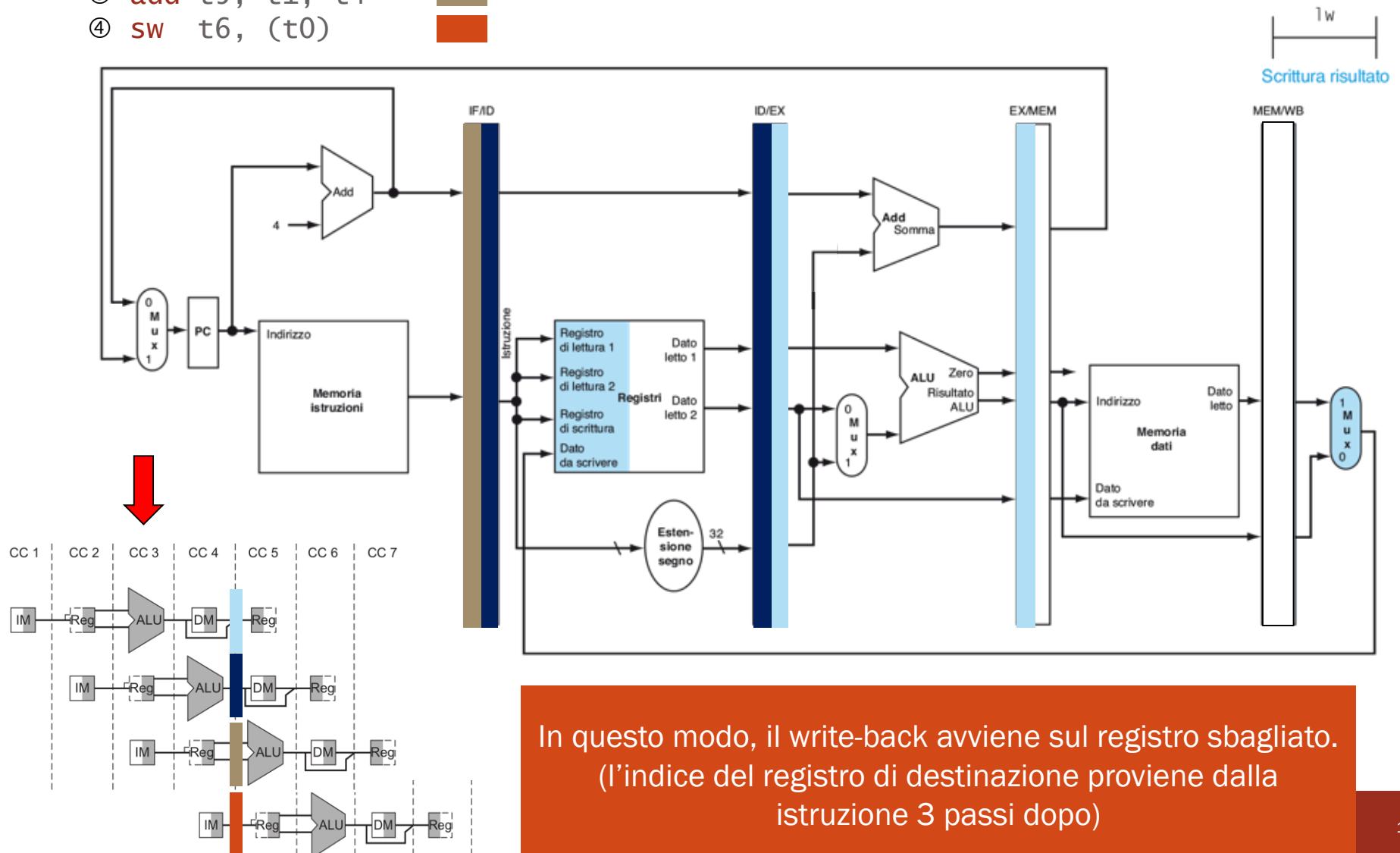
- ① **lw** t4, (t0)
- ② **add** t3, t1, t2
- ③ **add** t5, t1, t4
- ④ **sw** t6, (t0)



In questo modo, il write-back avviene sul registro sbagliato.
(l'indice del registro di destinazione proviene dalla istruzione 3 passi dopo)

Write Back e altre istruzioni

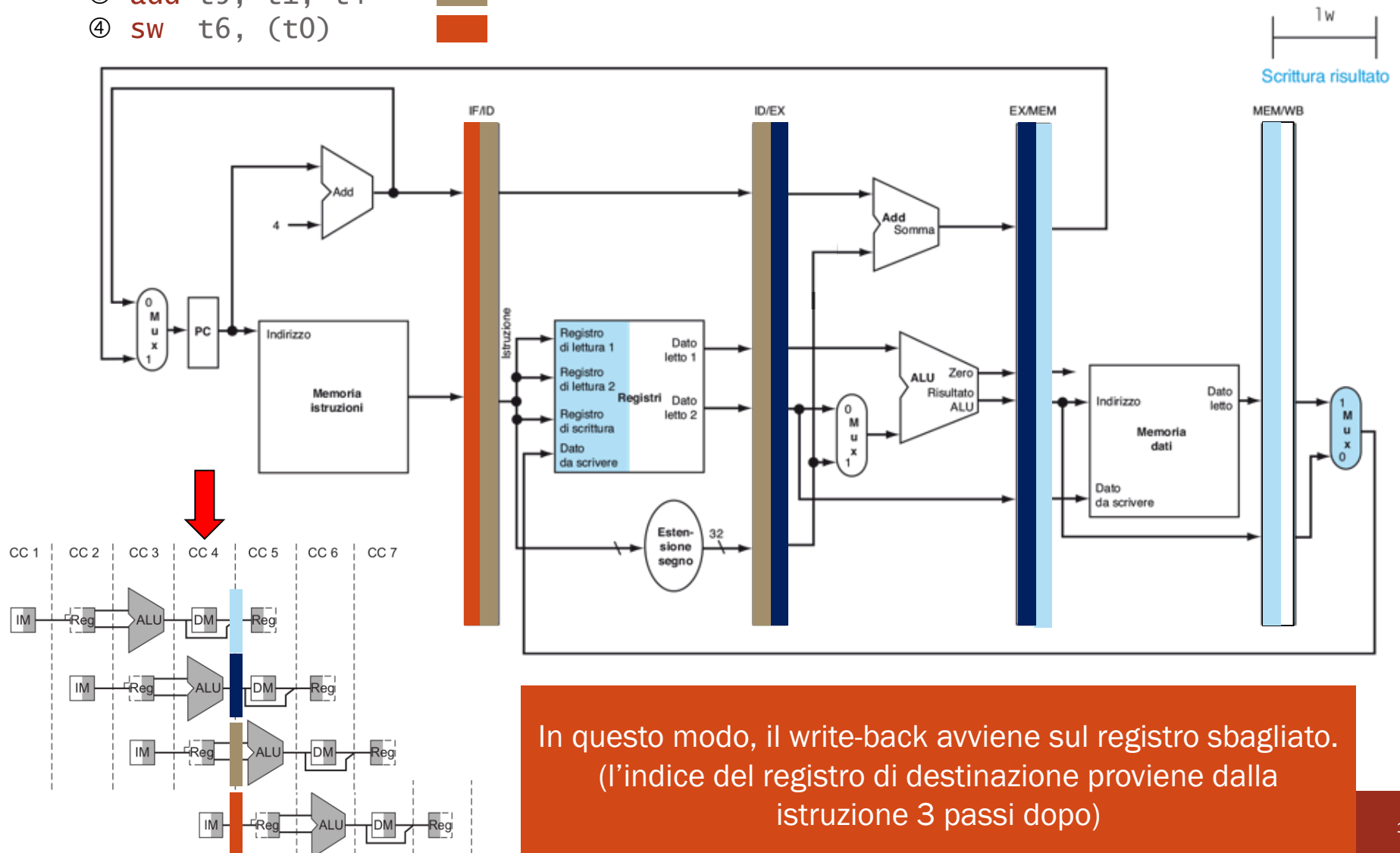
- ① **lw** t4, (t0)
- ② **add** t3, t1, t2
- ③ **add** t5, t1, t4
- ④ **sw** t6, (t0)



In questo modo, il write-back avviene sul registro sbagliato.
(l'indice del registro di destinazione proviene dalla
istruzione 3 passi dopo)

Write Back e altre istruzioni

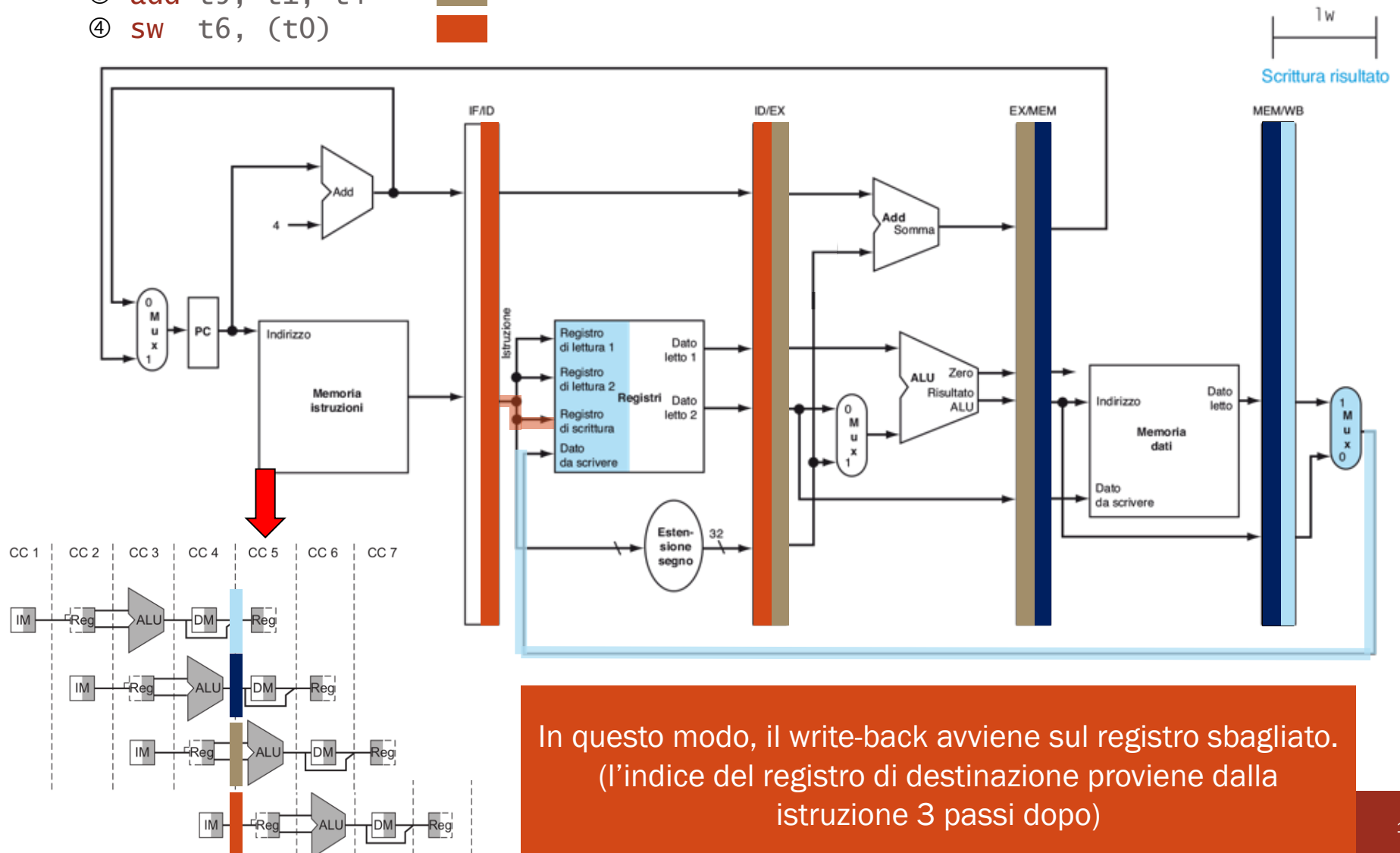
- ① **lw** t4, (t0)
- ② **add** t3, t1, t2
- ③ **add** t5, t1, t4
- ④ **sw** t6, (t0)



In questo modo, il write-back avviene sul registro sbagliato.
(l'indice del registro di destinazione proviene dalla
istruzione 3 passi dopo)

Write Back e altre istruzioni

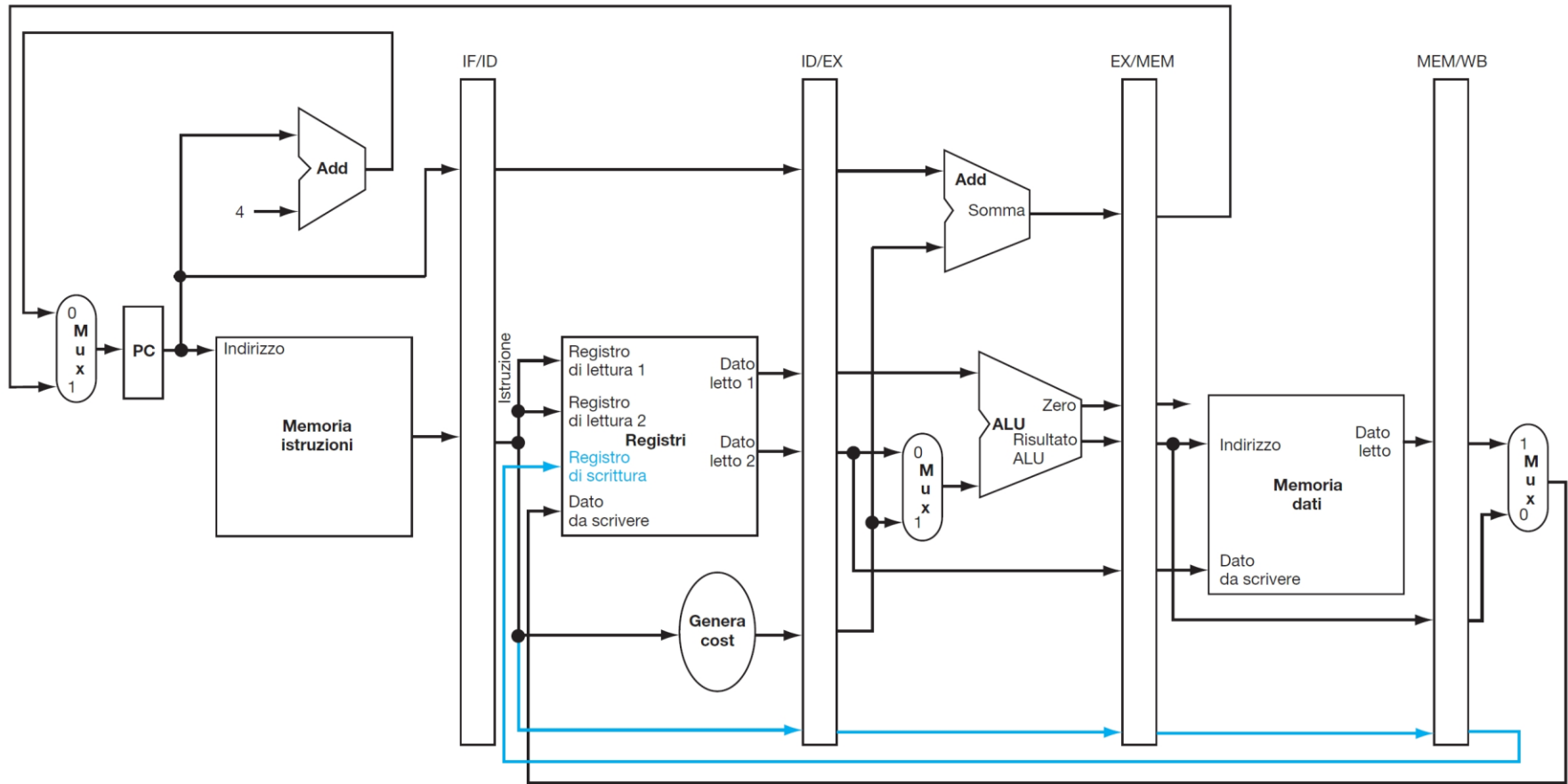
- ① **lw** t4, (t0)
- ② **add** t3, t1, t2
- ③ **add** t5, t1, t4
- ④ **sw** t6, (t0)



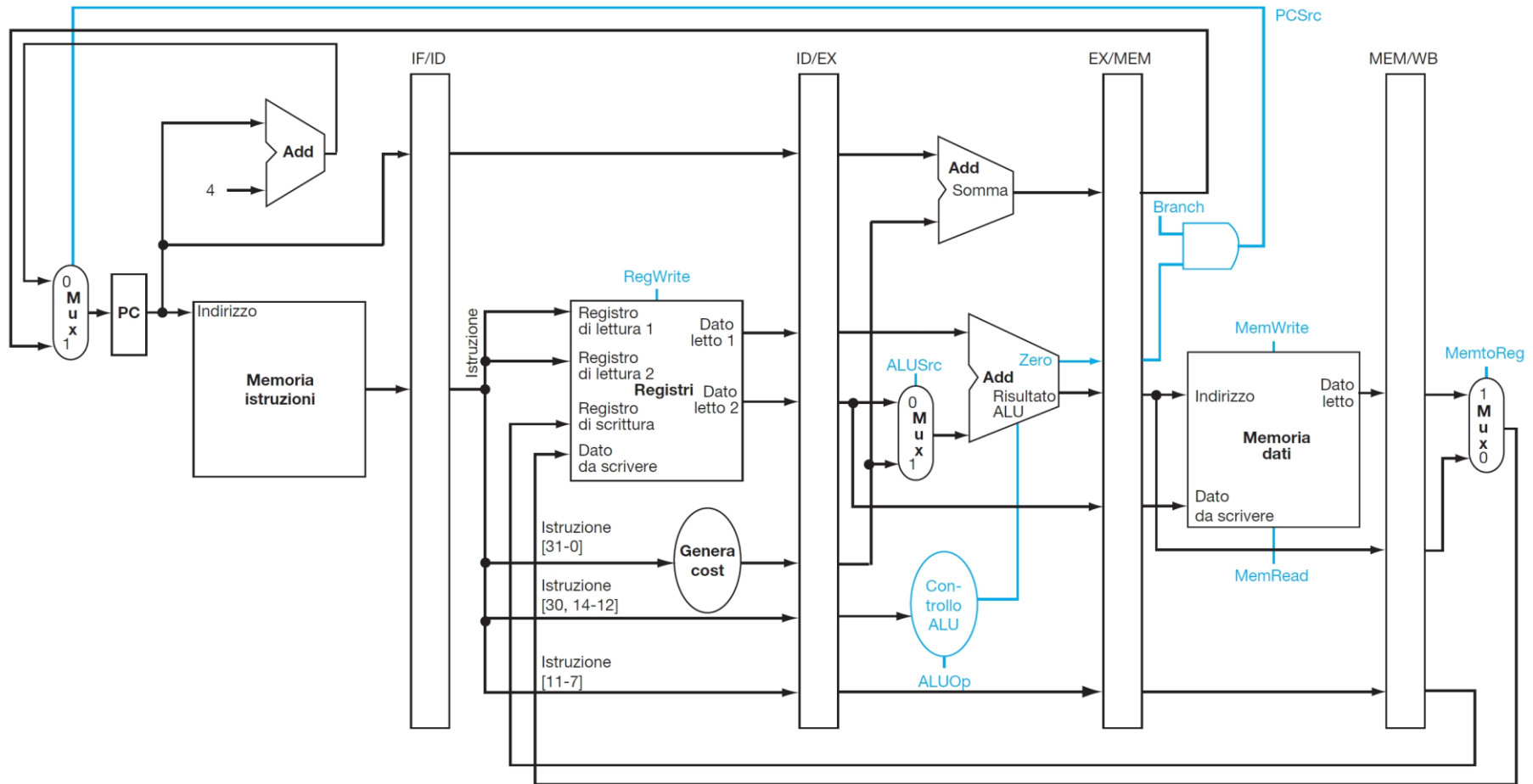
In questo modo, il write-back avviene sul registro sbagliato.
(l'indice del registro di destinazione proviene dalla
istruzione 3 passi dopo)

Write Back (corretto)

Tutte le informazioni ed i segnali di controllo devono essere nel registro precedente della pipeline



Con la logica dei salti (beq)



I segnali di controllo della CU

Possiamo dividere i segnali di controllo delle fasi esecutive in 3 gruppi:

fase EXE

fase MEM

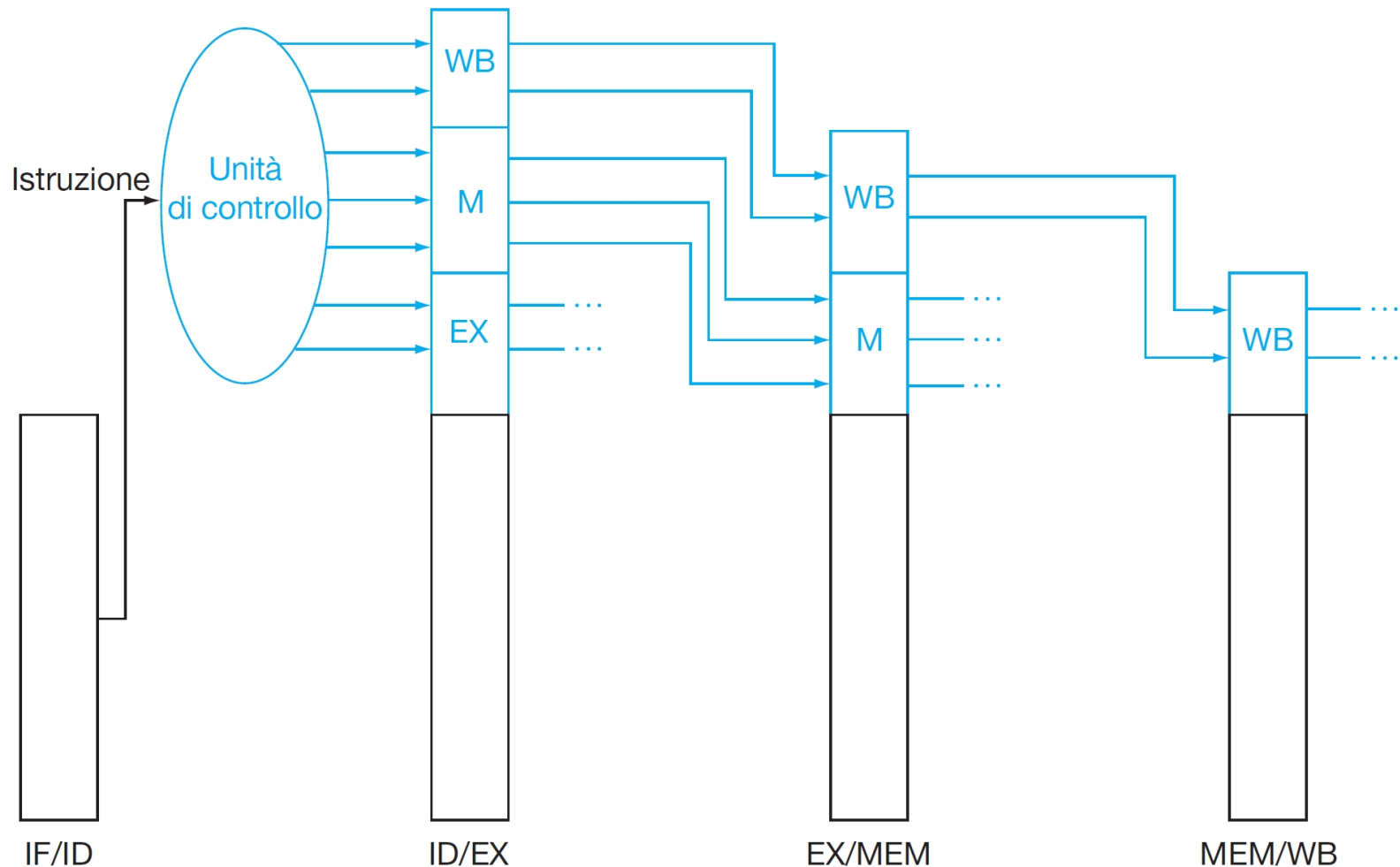
fase WB

Istruzione	Segnali di controllo dello stadio di esecuzione/calcolo dell'indirizzo		Segnali di controllo dello stadio di accesso alla memoria dati			Segnali di controllo dello stadio di scrittura	
	ALUOp	ALUSrc	Branch	MemRead	MemWrite	RegWrite	MemtoReg
Tipo R	10	0	0	0	0	1	0
lw	00	1	0	1	0	1	1
sw	00	1	0	0	1	0	X
beq	01	0	1	0	0	0	X

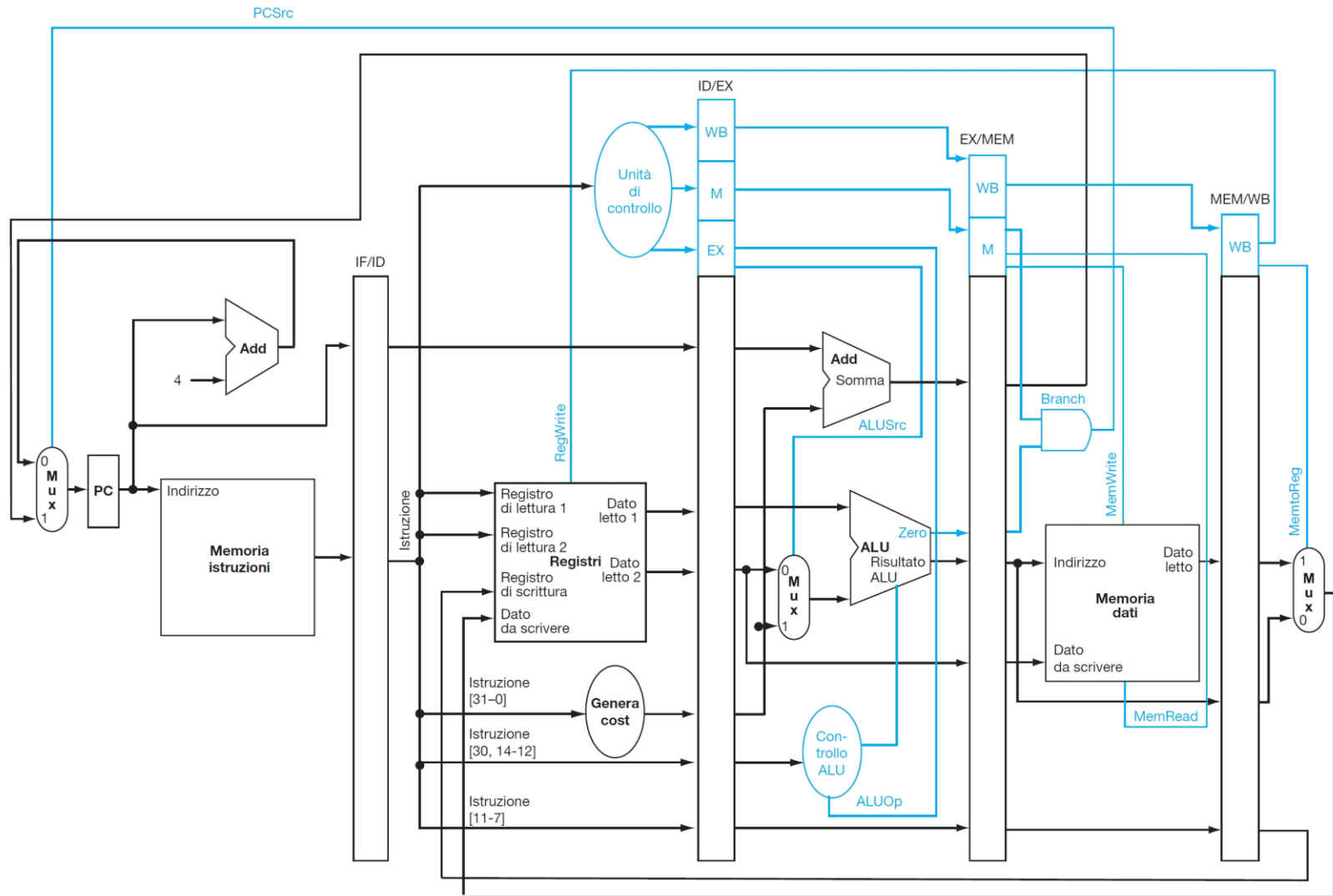
Questi segnali devono essere passati di fase in fase nei registri della pipeline

CU e registri della pipeline

Ad ogni fase salviamo i **dati** (letti dai registri o dalla parte immediata)
ed i **segnali di controllo** della stessa istruzione (generati dalla CU)



La CPU (quasi) completa



Data hazard e forwarding



SAPIENZA
UNIVERSITÀ DI ROMA

Controllo degli hazard (senza Forwarding)

Calcolare il numero di cicli necessari ad eseguire le istruzioni seguenti:

- individuare i **data e control hazard**
- per **determinare se con il forwarding possono essere risolti**, tracciare il diagramma temporale della pipeline
- determinare **quali non possono essere risolti e necessitano di stalli** (e quanti stalli)
- tenere conto del tempo necessario a **caricare la pipeline**

Assumere

- che la beq salti alla fine della fase EXE,
- che il salto beq non sia ritardato.

Somma un vettore di word

sommaVettore:

```
        addi t0, x0, 0      # accumul.
        addi t2,x0, 0xE00 # base
        addi t1,x0, 0xE28 # fine

ciclo: beq t2, t1, fine
        lw t3, (t2)
        add t0, t0, t3
        addi t2, t2, 4
        beq x0,x0, ciclo    # "jump"

fine:  addi a7, x0, 1
        add a0, x0, t0
        ecall
```

Controllo degli hazard senza forwarding (soluzione)

sommaVettore:

```
addi t0, x0, 0
```

```
# fine istruzione al 5° ciclo di clock
```

```
addi t2,x0, 0xE00
```

IF ID EX MM WB 6°

```
addi t1,x0, 0xE28
```

#	IF	ID	EX	MM	WB	
7°						

```
ciclo: beq t2, t1, fine
```

-> -> IF ID EX MM WB 10° 19° 28° ... 100°

lw t3, (t2)

# IF ID EX MM WB	11°	20°	29°
------------------	-----	-----	-----

```
add t0, t0, t3
```

#	->	->	IF	ID	EX	MM	WB			
								14°	23°	...

```
addi t2, t2, 4
```

# IF ID EX MM WB	15°	24°	...
------------------	-----	-----	-----

```
beq x0,x0, ciclo
```

#	IF	ID	EX	MM	WB	
						16° 25° ...

```
fine:      addi a7,x0, 1
```

```
# beq:  -> -> IF ID EX MM WB
```

```
add a0, x0, t0
```

104°

Per cui: **ecall**

105°

- occorrono **2 stalli** tra **addi** e **beq** (istruzioni i3 e i4, rispettivamente)
- il ciclo impiega 5 colpi di clock più **2 stalli** tra **lw** (i5) e **add** (i6) e **2 stalli** tra **beq** (i4) e **beq** (i8) (per salto i4 preso: qui il data hazard tra i7 e i4 non influisce perché introdurrebbe un solo stallo ma i due beq in sequenza ne introducono due)
- il control-hazard su **beq** (i4) alla fine del ciclo inserisce **2 stalli** (salto preso come per i8)

Totale: $4[\text{riemp. pipel.}] + 3 + 2 + 10 \cdot (5 + 2 + 2) + 1(\text{uscita ciclo}) + 3 + 2 = 105$ colpi di clock

Data hazard senza forwarding

- ① **sub** x2,x1,x3 # Registro x2 scritto da ①
- ② **and** x12,x2,x5 # Primo operando dipende da ①
- ③ **or** x13,x6,x2 # Secondo operando dipende da ①
- ④ **add** x14,x2,x2 # Primo e secondo operando dipendono da ①
- ⑤ **sw** x15,100(x2) # Base address dipende da ①

Hazard:

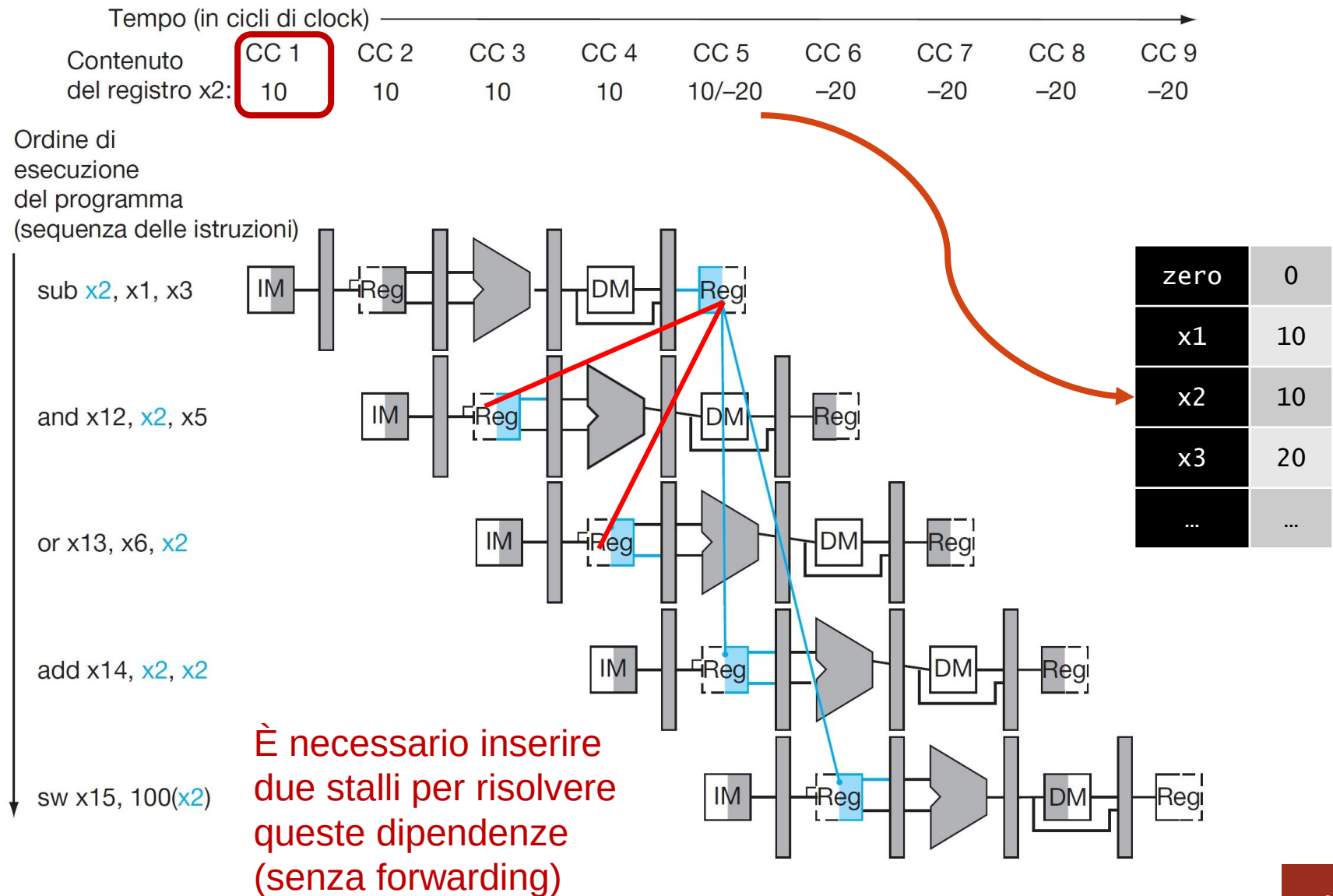
- ① **sub** x2,x1,x3 # IF ID EX MM **WB**
 - ② **and** x12,x2,x5 # IF **ID** EX MM WB
 - ③ **or** x13,x6,x2 # IF **ID** EX MM WB
 - ④ **add** x14,x2,x2 # IF **ID** EX MM WB
 - ⑤ **sw** x15,100(x2) # IF ID EX MM WB
- np: wb e ID nello stesso ciclo

zero	0
x1	10
x2	10
x3	20
...	...

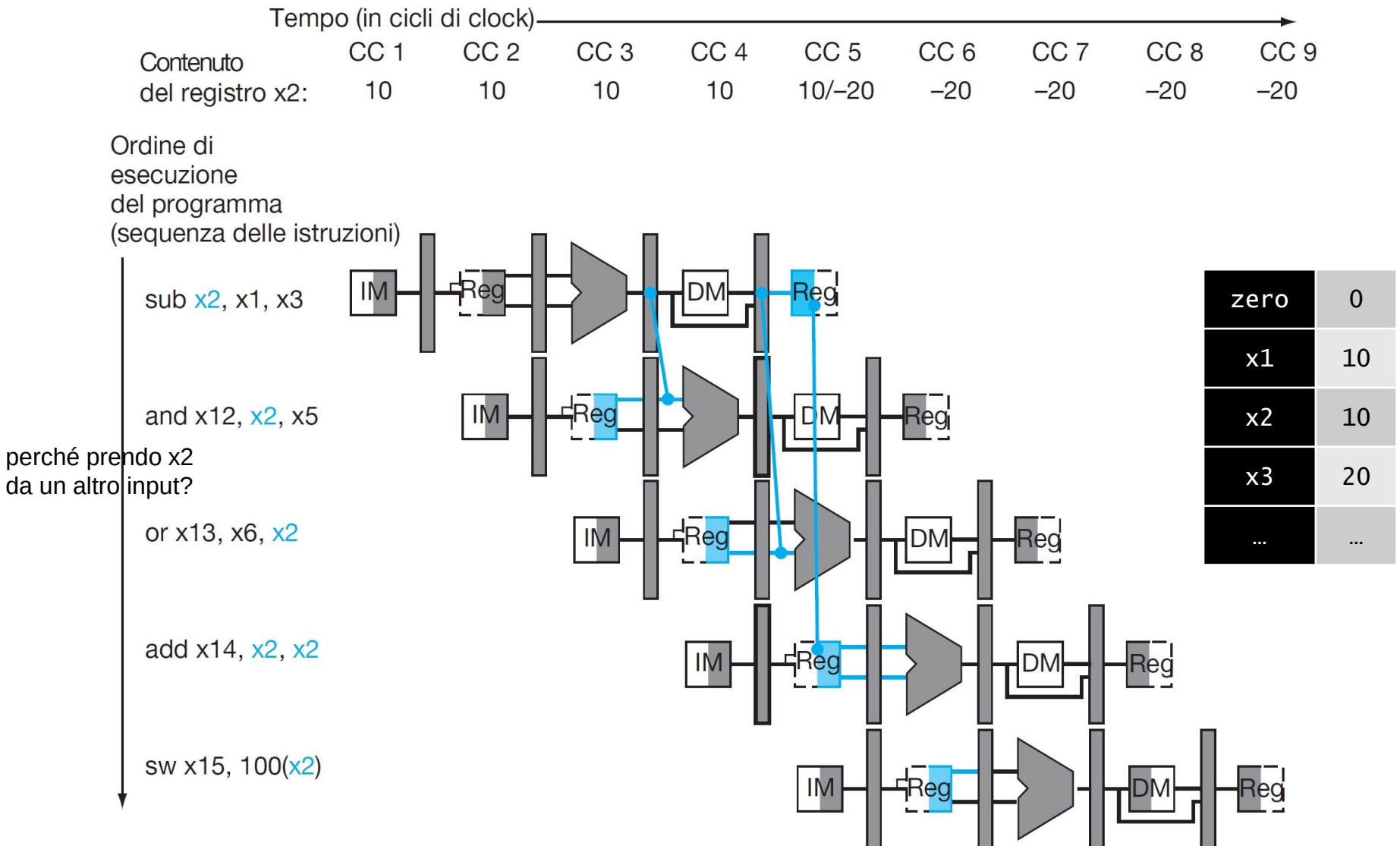
Con due fasi di stallo in ②:

- ① **sub** x2,x1,x3 # IF ID EX MM **WB**
- ② **and** x12,x2,x5 # -> -> IF **ID** EX MM WB
- ③ **or** x13,x6,x2 # IF **ID** EX MM WB
- ④ **add** x14,x2,x2 # IF **ID** EX MM WB
- ⑤ **sw** x15,100(x2) # IF ID EX MM WB

Data hazard senza forwarding



Data hazard con forwarding



non basta trasferire i registri nella pipeline: ad esempio la ALU deve eseguire l'istruzione definita due fasi prima (ma non sempre) quindi è necessario un'unità per compiere questa decisione!

Esercizio per casa: controllo degli hazard (con forwarding)

Calcolare il numero di cicli necessari ad eseguire le istruzioni seguenti:

- individuare i **data e control hazard**
- **per determinare se con il forwarding possono essere risolti**, tracciare il diagramma temporale della pipeline
- **determinare quali non possono essere risolti e necessitano di stalli** (e quanti stalli)
- tenere conto del tempo necessario a caricare la pipeline

Assumere

- che la beq salti alla fine della fase EXE,
- che il salto beq non sia ritardato.

sommaVettore:

```
addi t0, x0, 0
addi t2, x0, 0xE00
addi t1, x0, 0xE28
```

ciclo: beq t2, t1, fine

```
lw t3, (t2)
add t0, t0, t3
addi t2, t2, 4
```

```
beq x0, x0, ciclo
```

fine: addi a7, x0, 1
add a0, x0, t0
ecall



Esercizio per casa: cosa sarebbe successo con forwarding?